

LLM-Based C/C++ 1-Day Vulnerability Analysis Reporting System

Project Midterm Presentation

2026.05.12

사이버보안학과 노경민

AGENDA

Contents

01

Project Goal & RQ

문제 정의 · 연구 질문 · 방향 전환

02

Progress Summary

계획 대비 구현 단계 · 완료 기능

03

System Architecture

Extract · Triage · Finding · Verify · Report

04

Current Results & Demo

중간 결과 · 프로토타입 시연 흐름

05

AI Tool & Git Management

Claude/Codex 협업 · log.md · commit

06

Troubleshooting & Final Plan

기술 난제 · 5/30 Demo Day 일정

프로젝트 진화 & 연구 질문

Proposal → Reporting System · RQ · 설계 원칙

기존 방향

1-day Binary 자동 탐지

Target Open-source C/C++ binary
Source Magma benchmark + CVE pairs
Method Triage → Patch → BinaryMatch → Verify



현재 구현 방향

Source-level 취약점 분석 플랫폼

Target C/C++ source archive (ZIP 업로드)
Engine Claude Sonnet 4.6 multi-agent
Output Web UI + 자동 PDF 리포트

방향 전환 이유 실제 vulnerable/fixed binary artifact 확보가 지연되어, 먼저 dataset contract와 evidence-grounded report pipeline을 안정화하는 방향으로 전환

Research Question

C/C++ source 취약점에 대해, LLM이 의미적 추론과 근거 검증을 결합해
1-day 취약점 분석 리포트를 자동 생성할 수 있는가?

설계 원칙 · 검증되지 않은 LLM 출력은 reject한다.

Progress Summary

계획한 목표 대비 현재 구현 단계 및 완료된 기능

완료된 구현 단위

01	Dataset Contract	manifest.json · advisory · patch.diff · excerpt 구조 고정
02	Magma Importer	external/magma patch를 data/cases skeleton으로 변환
03	Evaluation Layer	detection / function localization / verifier 분포 출력
04	Source/ZIP Analyzer	C/C++ 파일 필터링, path traversal 방지, multi-agent 분석
05	Backend & Frontend	FastAPI API · React/Vite UI · progress stream · report viewer
06	Report Export	JSON · Markdown · PDF · llm_logs 저장

중간 산출물 Snapshot

139

Magma cases

demo 포함 skeleton 수

21

Tests passed

unittest + smoke test

3

Report formats

JSON / MD / PDF

5

Agent stages

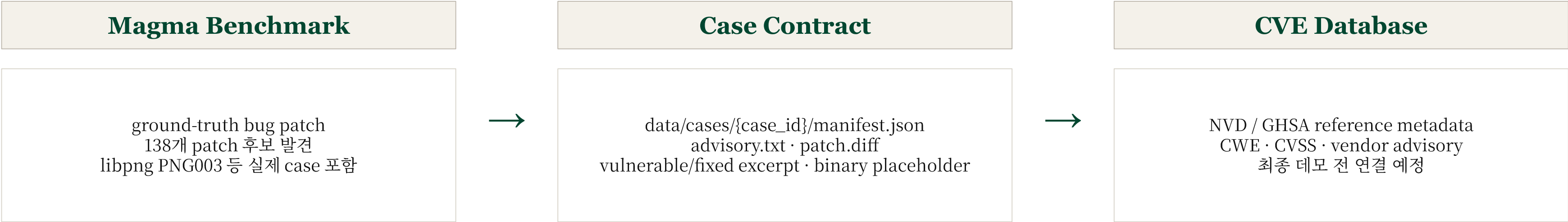
Extract→Report flow

현재 단계 상황

전체 뼈대를 만들어 놓은 상태, 동작은 하지만, 완벽하지 않음. Dataset과 성능 평가, 그리고 Policy 정제가 필요함.

Dataset & Evidence Sources

Magma 변환 및 CVE reference 계획, 그리고 현재 한계점



현재 데이터 구조

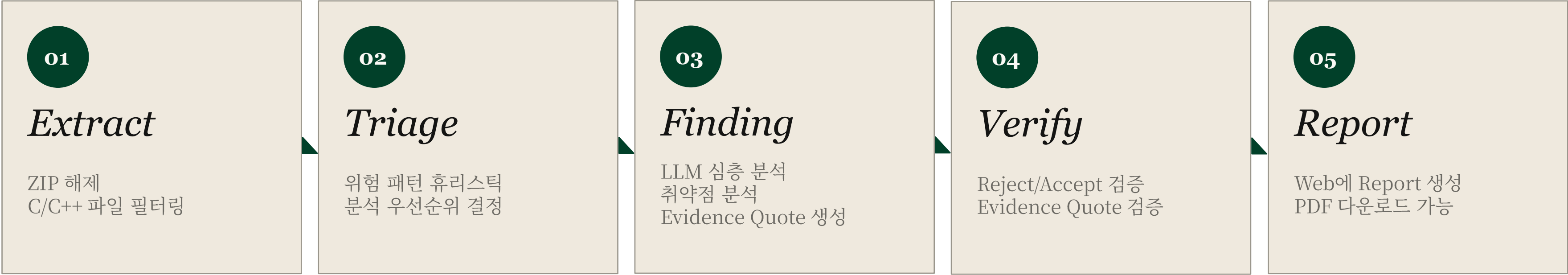
Magma patch를 곧바로 학습/평가 데이터로 쓰지 않고, 시스템이 읽을 수 있는 case directory로 정규화했다.
이 구조 덕분에 향후 실제 binary, decompiler output, CVE metadata를 추가해도 pipeline contract를 바꾸지 않아도 된다.

한계 및 수정 예정

대부분의 Magma case는 아직 실제 built binary가 아니라 source-level skeleton이다. 최종 발표 전에는 일부 case의 metadata를 보강하고, CVE reference와 실제 artifact 연결 범위를 명확히 제한해 발표한다. 추가적으로 현재는 libpng같은 작은 library에 한정되어 있으므로, 추후 추가 예정에 있다.

System Architecture

Extract → Triage → Finding → Verify → Report



구현상 핵심 변화

기존에는 binary를 본다고 했지만, C/C++의 소스 코드를 보는 것으로 수정하였다.
중간 구현에서는 source 입력을 기준으로 Finding과 Verify를 먼저 안정화해, 신뢰 가능한 report artifact를 만드는 데 집중했다.

Key Principle · LLM은 후보를 생성하고, verifier가 최종 보안 판단을 통제한다.

Extract

Unzip and Filtering

Zip 해제와 C/C++ 파일 필터링을 하는 과정

```
zip_analysis.py · extract()

def extract(zip_path) -> list[SourceFile]:
    sources, total = [], 0
    with zipfile.ZipFile(zip_path) as zf:
        for e in zf.infolist():
            # ① path traversal 차단
            if ".." in e.filename: continue
            # ② C/C++ 확장자만
            if not e.filename.endswith(ALLOWED):
                continue
            # ③ 크기 제한
            if e.file_size > MAX_FILE: continue
            if total+e.file_size > MAX_TOTAL: break
            if len(sources) >= MAX_FILES: break

            sources.append(SourceFile(...))
            total += e.file_size
    return sources
```

FILTER RULES

확장자: .c .cpp .cc .cxx .h .hpp
max_files: 20
max_file_size: 1 MB
max_total_bytes: 16 MB
path traversal: **REJECTED**
non-UTF-8: replaced (lossy)
사용자가 File Size를 정할 수 있도록 구성됨

EXAMPLE

Input 312 files (8.4 MB)
↓ extract
Output 20 C/C++ files

Triage

Dangerous Pattern Scanning

●●● source_analysis.py · HeuristicTriageAgent

```
DANGER_PATTERNS = {
    "buffer_overflow": r"\b(gets|strcpy|strcat|sprintf)\b"
    "unsafe_memcpy": r"\bmemcpy\s*\("
    "alloc_arithmetic": r"\b(malloc|calloc)\s*\([^)]*\)*"
    "unbounded_read": r"\b(read|recv|scanf)\s*\("
    "format_string": r"\bprintf\s*\(\s*\w+\s*\)"
}

def triage(source) -> TriageSignals:
    matches = []
    for kind, pat in DANGER_PATTERNS.items():
        for m in re.finditer(pat, source):
            line = source[:m.start()].count("\n") + 1
            matches.append((kind, line, m.group()))
    return TriageSignals(
        priority="high" if matches else "low"
        signals=matches,
    )
```

EXAMPLE OUTPUT · parse_header.c

```
buffer_overflow @ L23
    strcpy(buf, name)
unsafe_memcpy @ L47
    memcpy(dest, src, n)
alloc_arithmetic @ L12
    malloc(size * count)

priority: HIGH
```

WHY HEURISTIC?

✓

결정론 (같은 source → 같은 signal)

✓

정규식 휴리스틱으로 위험 패턴 사전 스캔

✓

Magma 데이터셋을 가지고 Heuristic한 패턴 구축

한계 및 수정 예정

현재는 정규식 휴리스틱을 통해서만 위험 패턴을 사전 스캔하는 과정이 있는데, 하지만 하드코딩 수준의 위험 패턴 사전에 그치므로 추후 CVE Database와 추가적인 Magma 데이터셋과 연결하여 위험 패턴을 분류 예정임. 현재는 LLM이 사용되지 않음.

Finding

Dangerous Pattern Scanning

ClaudeFindingAgent · prompt + API call

```
# === PROMPT ===
You are a security auditor reviewing
C/C++ source for memory corruption.
TRIAGE: {signals + line numbers}
SOURCE: {full file content}

Return ONLY JSON. CRITICAL:
evidence_quote must be EXACT substring.
Do not paraphrase. Do not invent code.

# === API CALL ===
resp = anthropic.messages.create(
    model="claude-sonnet-4-6"
    max_tokens=4096
    messages=[{"role": "user", ...}],
)
findings = json.loads(resp.content[0].text)
```

structured JSON response

```
{
  "findings": [{
    "vulnerability": "stack buffer overflow"
    "verdict": "VULNERABLE"
    "severity": "CRITICAL"
    "confidence": 0.94
    "line_start": 112, "line_end": 115
    "function": "parse_header"
    "evidence_quote":
      "strcpy(buf, user_data);"
    "root_cause":
      "fixed buf, no length check"
    "remediation":
      "strncpy(buf, src, sizeof(buf)-1)"
  }]
}
```

한계 및 수정 예정

현재는 단일 Claude Code Sonnet 4.6으로 API를 사용하고 있지만, 추후 제안 구조와 목표에 맞게 fine tuning 모델을 구축할 예정입니다.

Verification Layer & Report Artifact

Hallucination 방지를 위한 Rejection Policy와 결과물 구조

Reject Policy

- 1

evidence_quote가 비어 있으면 reject
- 2

quote가 실제 source 안에 없으면 reject
- 3

verdict가 vulnerable이 아니면 reject
- 4

confidence가 threshold (기본값 0.5) 보다 낮으면 reject
- 5

root cause / remediation이 없으면 reject
- 6

line reference가 source 범위를 벗어나면 reject

보안 판단 기준

모델의 자기 확신이 아니라 입력 artifact와 직접 연결되는 근거를 기준으로 accept/reject한다.
따라서 false positive를 줄이기 위해 의심스러운 결과는 과감히 NEEDS_REVIEW 또는 REJECT로 남긴다.

한계 및 수정 예정

현재는 confidenc를 기존 Claude로 생성하고 있기 때문에 confidence의 기준이 블랙박스이며 기준이 명확하지 않음. 추가적으로 Rejection Policy를 강화할 필요가 있음.
현재는 해당 코드만 보면 취약하지만 주변을 봤을 때 결국 취약하지 않은 것도 탐지하였지만 Rejection Policy를 추가하여 Reject을 한 사례가 있으므로 실제 Exploitable한 코드에 한정해야 함.

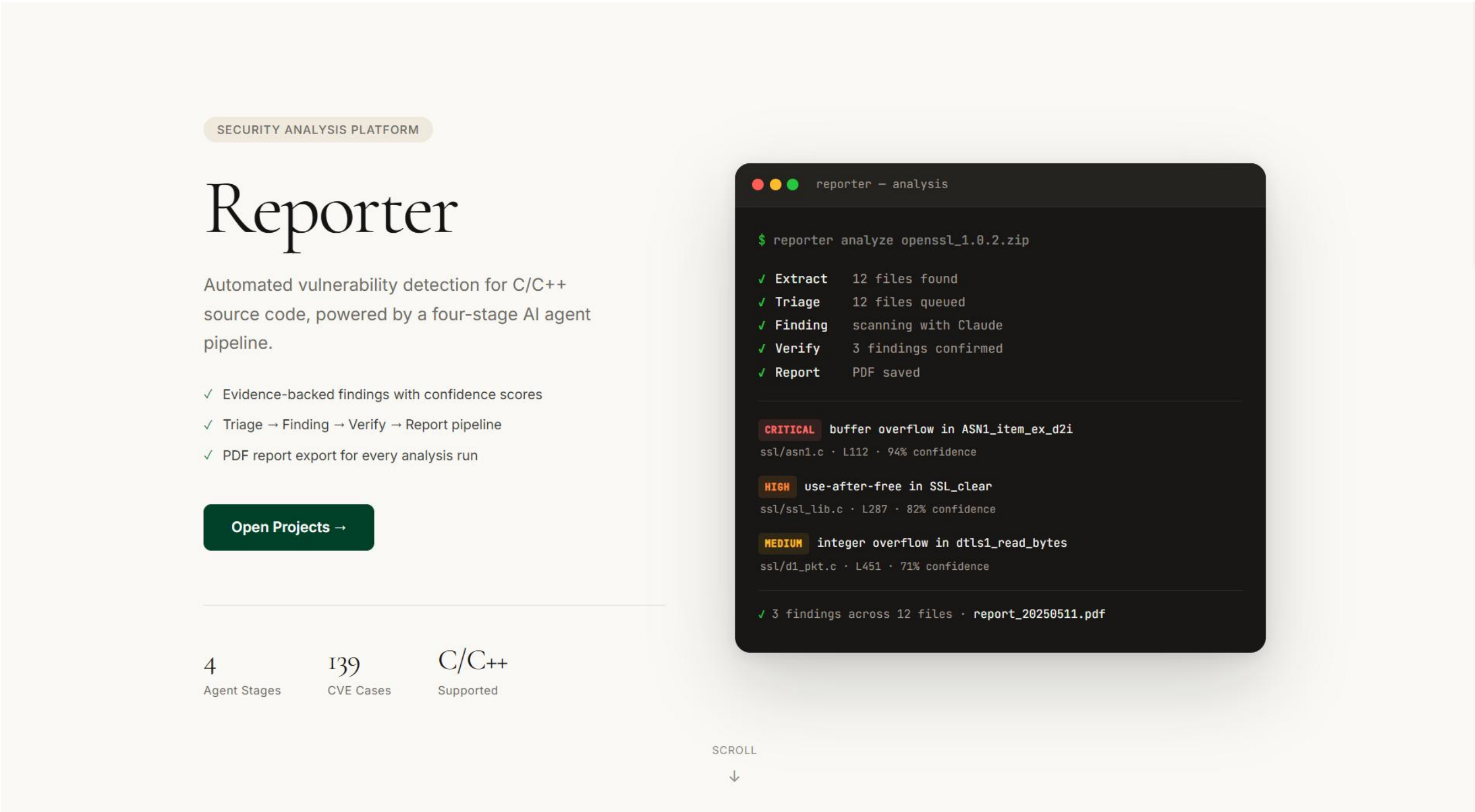
Report Artifact

report.json	machine-readable structured finding
report.md	human-readable analysis summary
report.pdf	presentation/report
llm_logs/*.md	agent별 판단 근거와 reject 사유 (XAI)

Current Results & Prototype Demo

중간 결과물 및 프로토타입 시연

Hero Landing Page



Current Results & Prototype Demo

중간 결과물 및 프로토타입 시연

Projects Page

Reporter

Projects

Projects

3 projects

+ New Project

Test 3

2026년 5월 12일 · 0 reports

test2

2026년 5월 11일 · 1 report

Pass

test

2026년 5월 11일 · 3 reports

Findings

Pass

Findings

Reporter

Projects

← Projects

Test 3

New Analysis

Drop ZIP file here

or click to browse

☐ Use heuristic mode (no API key needed)

Max files20

Start Analysis

Analysis History

No analyses yet. Upload a ZIP to start.

Current Results & Prototype Demo

중간 결과물 및 프로토타입 시연

Report Page

Reporter

Projects

← Project

Pass

example-libpng-master.zip

↓ Download PDF

Security Analysis Report

10 of 581 files analyzed

FINDINGS OVERVIEW

ACCEPTED

6

confirmed findings

REJECTED

2

filtered by verifier

FILES

IO

source files scanned

BY SEVERITY — ACCEPTED FINDINGS ONLY

2

HIGH

1

MEDIUM

3

LOW

SUMMARY

Analyzed 10 source files and found grounded findings in 3 files.

Files (10)

▶ example-libpng-master/arm/arm_init.c

Findings

0/0

Reporter

Projects

PNG_IMAGE_PIXEL_SIZE(image.format) does not overflow size_t before allocating. Additionally, check that the malloc return value is non-NULL and that the size passed is greater than zero.

▼ example-libpng-master/contrib/examples/simpleover.c

Pass2/4

Triage signals: sprintf, malloc. Accepted findings: 2. Rejected findings: 2.

ACCEPTED FINDINGS

High

Use of malloc'd buffer before NULL check

L442-446▲

simpleover_process()

95%

ROOT CAUSE

malloc() is called and the result is immediately passed to memset() before the NULL check. If malloc returns NULL, memset dereferences a null pointer, causing undefined behavior (crash).

EVIDENCE

```
sprites[nsprites].buffer = malloc(buf_size);
/* This buffer must be initialized to transparent: */
memset(sprites[nsprites].buffer, 0, buf_size);

if (sprites[nsprites].buffer != NULL)
```

REMEDIATION

Check the return value of malloc before calling memset. Move the NULL check immediately after malloc and only call memset if the pointer is non-NULL.

Medium

sprintf with fixed-size buffer may overflow

L426▼

REJECTED FINDINGS

High

PNG_IMAGE_SIZE integer overflow leading to under-allocated buffer

RejectedL223-225▼

High

PNG_IMAGE_SIZE integer overflow in main background buffer allocation

RejectedL554-556▼

AI Tool Management

인공지능과 어떻게 협업하였는가?

역할 분담



Human

범위 결정 · 보안 판단 기준 · prompt 설계 · 구현 결과 review



Claude Code

Frontend 제작 · React/Vite 화면 구성 · report viewer 구조



Codex

Backend pipeline · models · CLI/API · tests · packaging



프론트엔드 개발 예시

Design.md 사용



- Design.md는 Google Stitch에서 만든 LLM을 위한 Design 표준임.
- 디자이너 사이에서는 새로운 표준으로 사용되는 추세
- AI 기반 UI 개발에서 발생하는 디자인 일관성을 해소하기 위함.
- Design.md 파일을 개발 전에 먼저 참조하도록 하여 일관적인 UI를 만들 수 있었음.

AI Tool Management & Git Management

인공지능과 어떻게 협업하였는가? Git을 어떻게 사용하였는가?

implementation-log.md ↔ commit ↔ push 루틴

Implementation-log.md를 통해 프로젝트 구현 관리

WORKFLOW

commit → push 루틴

- 1 **의도 기록**
implementation-log.md 에 작업 의도/예상 변경 추가
- 2 **구현**
LLM과 코드 작성 · refactor
- 3 **테스트**
PYTHONPATH=src python3 -m unittest discover
- 4 **결과 기록**
변경 파일 · 검증 결과 · 남은 한계 추가 기록
- 5 **commit · push**
rohkyoungmin/AISECApp 원격 동기화

ROUTINE

- log.md 파일에 코드 구현 전 먼저 의도 기록
- LLM이 코드를 구현
- LLM이 짠 코드를 자동으로 테스트
- log.md 파일에 구현 결과를 기록
- Commit 후 Push
- LLM에게 코드 구현 과정의 Document를 같이 작성하게 하여, session이 끊기거나 다른 Agent를 사용할 때에도 이어서 코딩을 정확하게 할 수 있도록 Implementation Log를 문서화
- GitHub와 직접 연동하여 Commit과 Push를 할 수 있도록 권한 부여

Contributors 2

 **codex** Codex

 **claude** Claude

Git Management

깃허브 관리를 어떻게 하였는가?

rohkyoungmin/AISECApp – git log --oneline

c71083e	Add README and implementation docs	05.11
0aa6327	Redesign UI, tighten verifier, improve PDF	05.11
f931630	Add frontend project workflow	05.11
27ab8cd	Update default Claude model	05.11
4a4d624	Fix editable LLM install	05.11
f95c778	Refactor source analysis into multi-agent	05.11
4561353	Add ZIP source analysis backend	05.11
2f0ea2f	Add Claude source analysis report scaffold	05.11
0251ab7	Import Magma patch case skeletons	05.10
e323228	Initialize AISEC pipeline scaffold	05.10

Troubleshooting

기술적인 한계를 만났을 때 어떻게 해결했는가?

기술적 난제와 해결 과정

① WSL2 환경에서 npm 충돌

문제 Windows용 npm이 PATH 우선순위를 가져 \\wsl.localhost\... UNC 경로를 처리 못함

해결 nvm으로 Linux Node 20 LTS 별도 설치 + Windows node_modules 삭제 후 재설치

② Evidence Grounding 과도한 reject

문제 Claude가 ‘...’로 일부 생략하거나 공백이 다르면 단순 substring 매칭이 모두 reject

해결 ellipsis 기준 fragment 분리 + 12자 sliding window 매칭으로 grounding 알고리즘 교체

③ Exploitability 기준 모호

문제 evidence에 코드가 있어도 실제 exploit 가능 여부 판단 없이 이론적 취약점도 Accepted

해결 severity별 confidence 임계값 + dangerous op 필수 체크 도입 (HIGH/CRITICAL은 gets/strcpy 등 필수)

④ uvicorn --reload 미적용

문제 코드 변경 후 API 응답이 이전 코드 기준으로 계속 동작

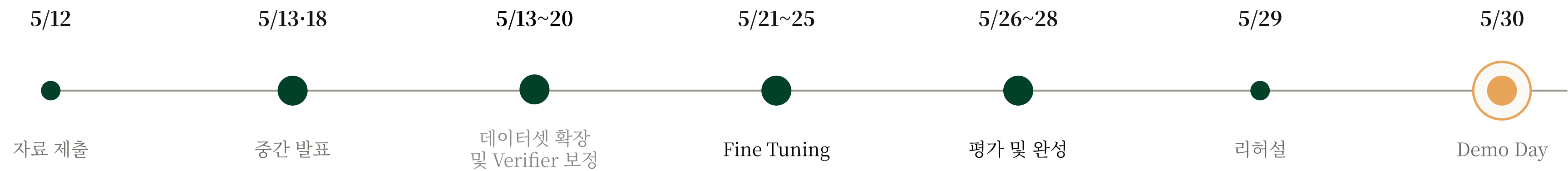
해결 --reload 옵션 추가 후 재시작 → StatReload가 소스 변경 자동 감지

Future Plan

추후 계획에 대하여

추후 계획

남은 2주 반 동안 정량 평가 · 실시간 스트리밍 · False Positive 보정 · 시연 리허설을 진행합니다.



Data

Dataset Expansion

- ☐ Magma libpng PNG003 외 케이스 확장
- ☐ CVE Database 연동

AI

Fine Tuning

- ☐ 인공지능 LLM Fine-Tuning
- ☐ 우리의 모델에 맞게 구성

Verify

Verifier Calibration

- ☐ Confidence 보정 curve
- ☐ Mitigation 감지 강화
- ☐ Exploitable 한 취약점만 Accept

Q&A

감사합니다